

# **BELAJAR PEMROGRAMAN WEB DENGAN BAHASA GO**

*Panduan Praktis dari Nol hingga Mahir*

Disusun dan diterjemahkan secara bebas ke dalam Bahasa Indonesia,  
berdasarkan materi tutorial pemrograman web Go  
dari situs *gowebexamples.com*

# Daftar Isi

- Bab 1.** Pendahuluan: Mengenal Bahasa Pemrograman Go
- Bab 2.** Hello World: Server Web Pertama Anda
- Bab 3.** HTTP Server: Menangani Permintaan dan File Statis
- Bab 4.** Routing dengan gorilla/mux
- Bab 5.** Bekerja dengan Basis Data MySQL
- Bab 6.** Template HTML
- Bab 7.** Aset dan File Statis
- Bab 8.** Membuat dan Memproses Form
- Bab 9.** Middleware Dasar
- Bab 10.** Middleware Tingkat Lanjut
- Bab 11.** Session dan Cookie
- Bab 12.** Bekerja dengan JSON
- Bab 13.** WebSocket
- Bab 14.** Hashing Kata Sandi dengan bcrypt

# Bab 1. Pendahuluan: Mengenal Bahasa Pemrograman Go

Go (sering disebut Golang) adalah bahasa pemrograman open source yang dirancang untuk membangun perangkat lunak yang sederhana, cepat, dan andal. Bahasa ini banyak dipakai oleh perusahaan-perusahaan besar untuk membangun layanan mereka karena performanya yang tinggi dan kemudahannya untuk dipelajari.

Buku ini disusun berdasarkan kumpulan tutorial dari situs Go Web Examples, yang berisi contoh-contoh kode singkat namun jelas tentang cara membangun aplikasi web menggunakan Go. Materinya mencakup dasar-dasar seperti membuat server HTTP, hingga topik yang lebih lanjut seperti routing, basis data, template, middleware, session, JSON, WebSocket, dan hashing kata sandi.

Salah satu keunggulan Go adalah ia sudah dilengkapi 'baterai' bawaan — artinya banyak fitur penting seperti web server sudah tersedia langsung di pustaka standarnya (standard library), tanpa perlu menginstal framework tambahan. Paket net/http misalnya, sudah mencakup hampir semua kebutuhan dasar untuk membuat server maupun klien HTTP.

## Apa yang Akan Anda Pelajari

Buku ini disusun secara bertahap, dari yang paling dasar hingga topik yang lebih kompleks. Setiap bab berisi penjelasan konsep, potongan kode contoh, dan kode lengkap yang siap dicoba (copy-paste) di komputer Anda.

- Membuat server web pertama dengan net/http
- Menangani permintaan dinamis dan menyajikan file statis
- Routing tingkat lanjut menggunakan gorilla/mux
- Menyimpan dan mengambil data dari basis data MySQL
- Menampilkan data dinamis dengan template HTML
- Membuat dan memproses form isian dari pengguna
- Middleware dasar dan lanjutan untuk logging serta validasi
- Menyimpan status login pengguna dengan session
- Mengolah data JSON
- Komunikasi dua arah secara real-time dengan WebSocket
- Mengamankan kata sandi pengguna dengan bcrypt

## Yang Anda Butuhkan Sebelum Memulai

Sebelum mempraktikkan contoh-contoh dalam buku ini, pastikan Go sudah terpasang di komputer Anda. Anda dapat memeriksanya dengan menjalankan perintah berikut di terminal:

```
go version
```

Jika Go belum terpasang, kunjungi situs resmi [golang.org](http://golang.org) untuk mengunduh dan memasangnya sesuai sistem operasi Anda. Setelah siap, mari kita mulai dengan contoh paling klasik: Hello World.

## Bab 2. Hello World: Server Web Pertama Anda

Go sudah dilengkapi dengan web server bawaan. Paket `net/http` dari pustaka standar berisi seluruh fungsi yang dibutuhkan untuk bekerja dengan protokol HTTP, termasuk klien HTTP dan server HTTP. Pada bab ini, Anda akan melihat betapa sederhananya membuat sebuah web server yang bisa diakses lewat browser.

### Mendaftarkan Request Handler

Langkah pertama adalah membuat sebuah handler, yaitu fungsi yang akan menerima semua koneksi HTTP masuk — baik dari browser, klien HTTP, maupun permintaan API. Sebuah handler di Go memiliki bentuk (signature) seperti berikut:

```
func (w http.ResponseWriter, r *http.Request)
```

Fungsi ini menerima dua parameter:

- `http.ResponseWriter` — tempat Anda menuliskan respons (teks/HTML) yang akan dikirim kembali ke pengguna.
- `http.Request` — berisi seluruh informasi tentang permintaan HTTP yang masuk, termasuk URL dan header.

Untuk mendaftarkan handler ke server HTTP bawaan, caranya cukup sederhana:

```
http.HandleFunc("/", func (w http.ResponseWriter, r *http.Request) {  
    fmt.Fprintf(w, "Hello, you've requested: %s\n", r.URL.Path)  
})
```

### Mendengarkan Koneksi HTTP

Handler saja belum bisa menerima koneksi HTTP dari luar. Server HTTP harus 'mendengarkan' (listen) pada sebuah port agar koneksi yang masuk dapat diteruskan ke handler. Karena port 80 adalah port standar untuk trafik HTTP, contoh ini akan menggunakan port tersebut.

Kode berikut akan menjalankan server HTTP bawaan Go dan mendengarkan koneksi pada port 80. Setelah dijalankan, Anda bisa membuka `http://localhost/` di browser untuk melihat server menangani permintaan Anda.

```
http.ListenAndServe(":80", nil)
```

### Kode Lengkap (Siap Disalin)

Berikut adalah kode lengkap yang bisa langsung Anda coba untuk mempraktikkan apa yang sudah dipelajari pada bab ini.

*main.go*

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, you've requested: %s\n", r.URL.Path)
    })

    http.ListenAndServe(":80", nil)
}
```

**Catatan:** Jalankan dengan perintah `go run main.go`, lalu buka `http://localhost/` di browser Anda. Karena port 80 kadang membutuhkan izin khusus di beberapa sistem operasi, Anda bisa menggantinya dengan port lain seperti `":8080"`.

## Bab 3. HTTP Server: Menangani Permintaan dan File Statis

Pada bab ini Anda akan belajar membuat HTTP server dasar yang lebih lengkap di Go. Sebuah server HTTP dasar umumnya memiliki tiga tugas utama:

- Memproses permintaan dinamis: menangani permintaan dari pengguna yang menjelajahi situs, login ke akun, atau mengunggah gambar.
- Menyajikan aset statis: menyajikan JavaScript, CSS, dan gambar ke browser untuk menciptakan pengalaman yang dinamis.
- Menerima koneksi: server HTTP harus mendengarkan pada port tertentu agar dapat menerima koneksi dari internet.

### Memproses Permintaan Dinamis

Paket `net/http` berisi semua utilitas yang dibutuhkan untuk menerima permintaan dan menanganinya secara dinamis. Kita bisa mendaftarkan handler baru dengan fungsi `http.HandleFunc`, yang parameter pertamanya adalah path yang ingin dicocokkan, dan parameter kedua adalah fungsi yang akan dijalankan.

```
http.HandleFunc("/", func (w http.ResponseWriter, r *http.Request) {  
    fmt.Fprint(w, "Welcome to my website!")  
})
```

Untuk aspek dinamisnya, `http.Request` menyimpan seluruh informasi tentang permintaan beserta parameternya. Anda bisa membaca parameter GET dengan `r.URL.Query().Get("token")` atau parameter POST (isian dari form HTML) dengan `r.FormValue("email")`.

### Menyajikan Aset Statis

Untuk menyajikan aset statis seperti JavaScript, CSS, dan gambar, kita memakai `http.FileServer` bawaan dan mengarahkannya ke sebuah path URL. Agar file server bisa bekerja, ia perlu tahu dari direktori mana file harus disajikan:

```
fs := http.FileServer(http.Dir("static/"))
```

Setelah file server siap, kita cukup mengarahkan sebuah path URL ke sana, sama seperti saat mendaftarkan handler dinamis. Satu hal penting: agar file tersaji dengan benar, kita perlu menghilangkan sebagian dari path URL — biasanya nama direktori tempat file kita berada.

```
http.Handle("/static/", http.StripPrefix("/static/", fs))
```

### Menerima Koneksi

Hal terakhir untuk melengkapi HTTP server dasar kita adalah mendengarkan pada sebuah port agar bisa menerima koneksi dari internet. Go juga memiliki server HTTP bawaan yang bisa langsung dijalankan:

```
http.ListenAndServe(":80", nil)
```

### Kode Lengkap (Siap Disalin)

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func (w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Welcome to my website!")
    })

    fs := http.FileServer(http.Dir("static/"))
    http.Handle("/static/", http.StripPrefix("/static/", fs))

    http.ListenAndServe(":80", nil)
}
```



## Bab 4. Routing dengan gorilla/mux

Paket `net/http` bawaan Go memang menyediakan banyak fungsi untuk protokol HTTP, tetapi satu hal yang kurang baik ditanganinya adalah routing yang kompleks, misalnya memecah sebuah URL menjadi beberapa parameter. Untungnya, ada paket populer bernama `gorilla/mux` yang dikenal luas di komunitas Go karena kualitas kodenya yang baik.

### Memasang Paket gorilla/mux

`gorilla/mux` adalah paket yang menyesuaikan diri dengan router bawaan Go. Paket ini kompatibel dengan bentuk handler standar Go, `func (w http.ResponseWriter, r *http.Request)`, sehingga bisa dipadukan dengan pustaka HTTP lain seperti `middleware`. Instal paketnya dengan perintah `go get`:

```
go get -u github.com/gorilla/mux
```

### Membuat Router Baru

Pertama-tama, buat router baru. Router inilah yang nantinya akan diteruskan sebagai parameter ke server, menerima seluruh koneksi HTTP, dan meneruskannya ke handler yang sudah Anda daftarkan.

```
r := mux.NewRouter()
```

### Mendaftarkan Handler

Setelah memiliki router baru, Anda bisa mendaftarkan handler seperti biasa. Bedanya, alih-alih memanggil `http.HandleFunc(...)`, Anda memanggil `HandleFunc` pada router Anda: `r.HandleFunc(...)`.

### Parameter URL

Kekuatan terbesar dari router `gorilla/mux` adalah kemampuannya mengekstrak segmen dari URL permintaan. Sebagai contoh, perhatikan URL berikut dalam aplikasi Anda:

```
/books/go-programming-blueprint/page/10
```

URL ini memiliki dua segmen dinamis: slug judul buku (`go-programming-blueprint`) dan nomor halaman (10). Agar handler dapat mencocokkan URL semacam ini, gantilah segmen dinamis tersebut dengan placeholder pada pola URL:

```
r.HandleFunc("/books/{title}/page/{page}", func(w http.ResponseWriter, r *http.Request) {  
    // ambil data buku  
    // arahkan ke halaman yang sesuai  
})
```

Langkah terakhir adalah mengambil data dari segmen-segmen tersebut. Paket ini menyediakan fungsi `mux.Vars(r)` yang menerima `http.Request` sebagai parameter dan mengembalikan sebuah map berisi segmen-segmen tadi.

```
func(w http.ResponseWriter, r *http.Request) {
    vars := mux.Vars(r)
    vars["title"] // slug judul buku
    vars["page"]  // halaman
}
```

## Mengatur Router pada Server HTTP

Pernah bertanya-tanya apa arti nil pada `http.ListenAndServe(":80", nil)`? Itu adalah parameter untuk router utama server HTTP. Secara default nilainya nil, yang berarti menggunakan router bawaan paket `net/http`. Untuk memakai router Anda sendiri, ganti nil dengan variabel router Anda, `r`.

```
http.ListenAndServe(":80", r)
```

## Kode Lengkap (Siap Disalin)

```
package main

import (
    "fmt"
    "net/http"

    "github.com/gorilla/mux"
)

func main() {
    r := mux.NewRouter()

    r.HandleFunc("/books/{title}/page/{page}", func(w http.ResponseWriter, r *http.Request) {
        vars := mux.Vars(r)
        title := vars["title"]
        page := vars["page"]

        fmt.Fprintf(w, "You've requested the book: %s on page %s\n", title, page)
    })

    http.ListenAndServe(":80", r)
}
```

## Fitur-Fitur Tambahan Router gorilla/mux

## Metode HTTP

Anda bisa membatasi handler hanya untuk metode HTTP tertentu:

```
r.HandleFunc("/books/{title}", CreateBook).Methods("POST")
r.HandleFunc("/books/{title}", ReadBook).Methods("GET")
r.HandleFunc("/books/{title}", UpdateBook).Methods("PUT")
r.HandleFunc("/books/{title}", DeleteBook).Methods("DELETE")
```

## Hostname dan Subdomain

Anda juga bisa membatasi handler hanya untuk hostname atau subdomain tertentu:

```
r.HandleFunc("/books/{title}", BookHandler).Host("www.mybookstore.com")
```

## Skema (HTTP/HTTPS)

Handler dapat dibatasi hanya untuk skema http atau https:

```
r.HandleFunc("/secure", SecureHandler).Schemes("https")
r.HandleFunc("/insecure", InsecureHandler).Schemes("http")
```

## Path Prefix dan Subrouter

Anda dapat membatasi handler pada prefix path tertentu, sehingga memudahkan pengelompokan route yang berkaitan:

```
bookrouter := r.PathPrefix("/books").Subrouter()
bookrouter.HandleFunc("/", AllBooks)
bookrouter.HandleFunc("/{title}", GetBook)
```

## Bab 5. Bekerja dengan Basis Data MySQL

Pada suatu titik, aplikasi web Anda pasti perlu menyimpan dan mengambil data dari sebuah basis data. Hal ini hampir selalu terjadi ketika Anda berurusan dengan konten dinamis, menyajikan form untuk diisi pengguna, atau menyimpan kredensial login. Untuk keperluan inilah basis data digunakan.

Basis data hadir dalam berbagai bentuk. Salah satu yang paling umum digunakan di seluruh dunia web adalah MySQL. Basis data ini sudah ada sejak lama dan telah membuktikan kestabilannya berkali-kali. Pada bab ini kita akan mempelajari dasar-dasar akses basis data di Go: membuat tabel, menyimpan data, dan mengambilnya kembali.

### Memasang Driver `go-sql-driver/mysql`

Go menyediakan paket bernama `database/sql` untuk melakukan query ke berbagai jenis basis data SQL. Paket ini berguna karena mengabstraksi fitur-fitur umum SQL ke dalam satu API. Yang tidak disertakan Go adalah driver basis data itu sendiri — driver adalah paket yang mengimplementasikan detail teknis dari basis data tertentu (dalam kasus ini, MySQL).

Untuk memasang driver MySQL, jalankan perintah berikut di terminal:

```
go get -u github.com/go-sql-driver/mysql
```

### Menghubungkan ke Basis Data MySQL

Hal pertama yang perlu dipastikan setelah memasang paket-paket yang dibutuhkan adalah apakah kita bisa terhubung ke server MySQL dengan sukses. Jika Anda belum memiliki server MySQL yang berjalan, Anda bisa menjalankannya dengan mudah menggunakan Docker (lihat dokumentasi resmi image Docker MySQL di [hub.docker.com](https://hub.docker.com)).

Untuk memeriksa koneksi, impor paket `database/sql` dan `go-sql-driver/mysql`, lalu buka koneksi seperti berikut:

```
import "database/sql"
import _ "go-sql-driver/mysql"

// Konfigurasi koneksi basis data (selalu periksa error)
db, err := sql.Open("mysql", "username:password@(127.0.0.1:3306)/dbname?parseTime=true")

// Uji koneksi pertama ke basis data untuk memastikan semuanya berjalan lancar.
// Jangan lupa periksa error-nya.
err := db.Ping()
```

### Membuat Tabel Pertama

Setiap entri data pada basis data disimpan dalam sebuah tabel tertentu. Tabel basis data terdiri dari kolom dan baris. Kolom memberi label pada setiap entri data dan menentukan tipenya, sedangkan baris adalah nilai data yang dimasukkan. Pada contoh pertama, kita ingin membuat tabel seperti berikut:

| id | username | password | created_at          |
|----|----------|----------|---------------------|
| 1  | johndoe  | secret   | 2019-08-10 12:30:00 |

Jika diterjemahkan ke SQL, perintah untuk membuat tabel di atas terlihat seperti ini:

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT,  
  username TEXT NOT NULL,  
  password TEXT NOT NULL,  
  created_at DATETIME,  
  PRIMARY KEY (id)  
);
```

Setelah perintah SQL siap, kita bisa menggunakan paket database/sql untuk membuat tabel tersebut di basis data MySQL kita:

```
query := `  
  CREATE TABLE users (  
    id INT AUTO_INCREMENT,  
    username TEXT NOT NULL,  
    password TEXT NOT NULL,  
    created_at DATETIME,  
    PRIMARY KEY (id)  
  );`  
  
// Menjalankan query SQL pada basis data. Periksa err untuk memastikan tidak ada error.  
_, err := db.Exec(query)
```

## Memasukkan Data Pengguna Pertama

Jika Anda sudah familier dengan SQL, memasukkan data baru ke tabel sama mudahnya dengan membuat tabel. Satu hal penting: secara default, Go menggunakan prepared statement untuk memasukkan data dinamis ke dalam query SQL — cara yang aman untuk meneruskan data dari pengguna ke basis data tanpa risiko serangan. Di masa awal pemrograman web, banyak programmer meneruskan data langsung bersama query ke basis data, yang menyebabkan kerentanan besar dan dapat merusak seluruh aplikasi web. Jangan lakukan itu — untungnya di Go, melakukannya dengan benar itu mudah.

Untuk memasukkan pengguna pertama ke tabel kita, buat query SQL seperti berikut. Perhatikan bahwa kolom id sengaja dilewatkan karena akan diisi otomatis oleh MySQL. Tanda tanya (?) memberi tahu

driver SQL bahwa itu adalah placeholder untuk data sebenarnya — inilah wujud prepared statement yang dimaksud.

```
INSERT INTO users (username, password, created_at) VALUES (?, ?, ?)
```

Kita bisa memakai query ini di Go untuk memasukkan baris baru ke tabel:

```
import "time"

username := "johndoe"
password := "secret"
createdAt := time.Now()

// Memasukkan data ke tabel users dan mengembalikan hasil beserta kemungkinan error.
// Hasilnya berisi informasi tentang id terakhir yang dibuat (otomatis) dan jumlah baris
yang terpengaruh.
result, err := db.Exec(`INSERT INTO users (username, password, created_at) VALUES (?, ?,
?)`, username, password, createdAt)
```

Untuk mengambil id yang baru dibuat untuk pengguna Anda, cukup lakukan seperti ini:

```
userID, err := result.LastInsertId()
```

## Mengambil Data Satu Pengguna

Setelah ada pengguna di tabel, kita ingin mengambilnya kembali beserta seluruh informasinya. Di Go, ada dua cara untuk melakukan query: `db.Query` untuk mengambil banyak baris sekaligus (untuk kemudian di-iterasi), dan `db.QueryRow` jika kita hanya ingin mengambil satu baris tertentu.

Query SQL untuk mengambil satu pengguna berdasarkan ID-nya terlihat seperti ini:

```
SELECT id, username, password, created_at FROM users WHERE id = ?
```

Di Go, kita mendeklarasikan beberapa variabel untuk menampung data, lalu meng-query satu baris basis data seperti berikut:

```
var (
    id          int
    username    string
    password    string
    createdAt   time.Time
)

// Query basis data dan simpan hasilnya ke variabel-variabel di atas. Jangan lupa periksa
error.
query := `SELECT id, username, password, created_at FROM users WHERE id = ?`
err := db.QueryRow(query, 1).Scan(&id, &username, &password, &createdAt)
```

## Mengambil Seluruh Data Pengguna

Pada bagian sebelumnya kita sudah mempelajari cara meng-query satu baris pengguna. Banyak aplikasi juga membutuhkan cara untuk mengambil seluruh data pengguna sekaligus. Caranya mirip dengan contoh di atas, hanya sedikit lebih banyak kodenya.

Kita bisa memakai query SQL yang sama seperti sebelumnya, tanpa klausa WHERE, sehingga kita mengambil seluruh pengguna yang ada:

```
SELECT id, username, password, created_at FROM users
```

Berikut cara mengambil seluruh baris dan menampungnya ke dalam sebuah slice:

```
type user struct {
    id          int
    username    string
    password    string
    createdAt   time.Time
}

rows, err := db.Query(`SELECT id, username, password, created_at FROM users`) // periksa err
defer rows.Close()

var users []user
for rows.Next() {
    var u user
    err := rows.Scan(&u.id, &u.username, &u.password, &u.createdAt) // periksa err
    users = append(users, u)
}
err := rows.Err() // periksa err
```

Isi slice users kurang lebih akan terlihat seperti berikut:

```
users {
  user {
    id:          1,
    username:    "johndoe",
    password:    "secret",
    createdAt:   time.Time{wall: 0x0, ext: 63701044325, loc: (*time.Location)(nil)},
  },
  user {
    id:          2,
    username:    "alice",
    password:    "bob",
    createdAt:   time.Time{wall: 0x0, ext: 63701044622, loc: (*time.Location)(nil)},
  },
}
```

## Menghapus Data Pengguna

Terakhir, menghapus pengguna dari tabel sama sederhananya dengan `.Exec` pada bagian-bagian sebelumnya:

```
_ , err := db.Exec(`DELETE FROM users WHERE id = ?`, 1) // periksa err
```

## Kode Lengkap (Siap Disalin)

```
package main

import (
    "database/sql"
    "fmt"
    "log"
    "time"

    _ "github.com/go-sql-driver/mysql"
)

func main() {
    db, err := sql.Open("mysql", "root:root@(127.0.0.1:3306)/root?parseTime=true")
    if err != nil {
        log.Fatal(err)
    }
    if err := db.Ping(); err != nil {
        log.Fatal(err)
    }

    { // Membuat tabel baru
        query := `
            CREATE TABLE users (
                id INT AUTO_INCREMENT,
                username TEXT NOT NULL,
                password TEXT NOT NULL,
                created_at DATETIME,
                PRIMARY KEY (id)
            );`

        if _, err := db.Exec(query); err != nil {
            log.Fatal(err)
        }
    }

    { // Memasukkan pengguna baru
        username := "johndoe"
        password := "secret"
        createdAt := time.Now()

        result, err := db.Exec(`INSERT INTO users (username, password, created_at) VALUES
        (?, ?, ?)`, username, password, createdAt)
        if err != nil {
```



```

        log.Fatal(err)
    }

    id, err := result.LastInsertId()
    fmt.Println(id)
}

{ // Mengambil satu pengguna
    var (
        id      int
        username string
        password string
        createdAt time.Time
    )

    query := "SELECT id, username, password, created_at FROM users WHERE id = ?"
    if err := db.QueryRow(query, 1).Scan(&id, &username, &password, &createdAt); err !=
nil {
        log.Fatal(err)
    }

    fmt.Println(id, username, password, createdAt)
}

{ // Mengambil seluruh pengguna
    type user struct {
        id      int
        username string
        password string
        createdAt time.Time
    }

    rows, err := db.Query(`SELECT id, username, password, created_at FROM users`)
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

    var users []user
    for rows.Next() {
        var u user

        err := rows.Scan(&u.id, &u.username, &u.password, &u.createdAt)
        if err != nil {
            log.Fatal(err)
        }
        users = append(users, u)
    }
    if err := rows.Err(); err != nil {
        log.Fatal(err)
    }

    fmt.Printf("%#v", users)
}

```

```
{
    _, err := db.Exec(`DELETE FROM users WHERE id = ?`, 1)
    if err != nil {
        log.Fatal(err)
    }
}
```

## Bab 6. Template HTML

Paket `html/template` milik Go menyediakan bahasa templating yang kaya fitur untuk membuat halaman HTML. Paket ini banyak dipakai pada aplikasi web untuk menampilkan data secara terstruktur di browser pengguna. Salah satu keunggulan besar dari bahasa templating Go adalah proses escaping data yang dilakukan secara otomatis — Anda tidak perlu khawatir tentang serangan XSS, karena Go akan mem-parsing template HTML dan meng-escape seluruh input sebelum ditampilkan ke browser.

### Template Pertama Anda

Menulis template di Go sangat sederhana. Contoh berikut menampilkan daftar TODO dalam bentuk unordered list (`ul`) HTML. Data yang diteruskan ke template bisa berupa struktur data apa pun milik Go — bisa berupa string atau angka sederhana, bahkan struktur data bertingkat seperti contoh di bawah ini. Untuk mengakses data pada template, variabel paling atas diakses dengan `{{.}}`. Titik di dalam kurung kurawal ini disebut pipeline, yaitu elemen akar dari data.

```
data := TodoPageData{
    PageTitle: "My TODO list",
    Todos: []Todo{
        {Title: "Task 1", Done: false},
        {Title: "Task 2", Done: true},
        {Title: "Task 3", Done: true},
    },
}
```

```
<h1>{{.PageTitle}}</h1>
<ul>
    {{range .Todos}}
        {{if .Done}}
            <li class="done">{{.Title}}</li>
        {{else}}
            <li>{{.Title}}</li>
        {{end}}
    {{end}}
</ul>
```

### Struktur Kontrol dalam Template

Bahasa templating Go memiliki sejumlah struktur kontrol yang kaya untuk menampilkan HTML Anda. Berikut ringkasan struktur yang paling sering digunakan. Untuk daftar lengkapnya, Anda bisa mengunjungi dokumentasi resmi paket `text/template`.

| Struktur Kontrol                | Keterangan                                       |
|---------------------------------|--------------------------------------------------|
| <code>{{/* komentar */}}</code> | Mendefinisikan sebuah komentar                   |
| <code>{{.}}</code>              | Menampilkan elemen akar (root)                   |
| <code>{{.Title}}</code>         | Menampilkan field 'Title' pada elemen bertingkat |

|                                             |                                                                                               |
|---------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>{{if .Done}} {{else}} {{end}}</code>  | Mendefinisikan pernyataan if                                                                  |
| <code>{{range .Todos}} {{.}} {{end}}</code> | Melakukan perulangan atas seluruh 'Todos' dan menampilkan tiap item dengan <code>{{.}}</code> |
| <code>{{block "content" .}} {{end}}</code>  | Mendefinisikan sebuah block bernama 'content'                                                 |

## Mem-parsing Template dari Berkas

Template dapat di-parsing baik dari string maupun dari berkas di disk. Karena umumnya template di-parsing dari berkas, contoh ini menunjukkan caranya. Pada contoh ini, ada sebuah berkas template di direktori yang sama dengan program Go bernama `layout.html`.

```
tmpl, err := template.ParseFiles("layout.html")
// atau
tmpl := template.Must(template.ParseFiles("layout.html"))
```

## Menjalankan Template pada Request Handler

Setelah template di-parsing dari disk, template siap dipakai di dalam request handler. Fungsi `Execute` menerima sebuah `io.Writer` untuk menuliskan hasil template dan sebuah `interface{}` untuk meneruskan data ke template. Ketika fungsi ini dipanggil pada sebuah `http.ResponseWriter`, header `Content-Type` pada respons HTTP akan otomatis diatur menjadi `text/html; charset=utf-8`.

```
func(w http.ResponseWriter, r *http.Request) {
    tmpl.Execute(w, "data goes here")
}
```

## Kode Lengkap (Siap Disalin)

*main.go*

```
package main

import (
    "html/template"
    "net/http"
)

type Todo struct {
    Title string
    Done bool
}

type TodoPageData struct {
    PageTitle string
    Todos      []Todo
}

func main() {
```

```

tmpl := template.Must(template.ParseFiles("layout.html"))
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    data := TodoPageData{
        PageTitle: "My TODO list",
        Todos: []Todo{
            {Title: "Task 1", Done: false},
            {Title: "Task 2", Done: true},
            {Title: "Task 3", Done: true},
        },
    }
    tmpl.Execute(w, data)
})
http.ListenAndServe(":80", nil)
}

```

*layout.html*

```

<h1>{{.PageTitle}}</h1>
<ul>
    {{range .Todos}}
        {{if .Done}}
            <li class="done">{{.Title}}</li>
        {{else}}
            <li>{{.Title}}</li>
        {{end}}
    {{end}}
</ul>

```

## Bab 7. Aset dan File Statis

Bab ini menunjukkan cara menyajikan file statis seperti CSS, JavaScript, atau gambar dari sebuah direktori tertentu.

*static-files.go*

```
package main

import "net/http"

func main() {
    fs := http.FileServer(http.Dir("assets/"))
    http.Handle("/static/", http.StripPrefix("/static/", fs))

    http.ListenAndServe(":8080", nil)
}
```

Misalkan struktur direktori assets Anda terlihat seperti berikut:

```
$ tree assets/
assets/
├── css
│   └── styles.css
```

Setelah dijalankan, Anda bisa mengakses file CSS tadi melalui URL `/static/css/styles.css`:

```
$ go run static-files.go

$ curl -s http://localhost:8080/static/css/styles.css
body {
    background-color: black;
}
```

**Catatan:** Perhatikan bahwa path pada `http.StripPrefix` harus sama dengan path yang didaftarkan pada `http.Handle`. Ini diperlukan agar Go tahu bagian mana dari URL yang harus 'dibuang' sebelum mencari file di direktori assets.

## Bab 8. Membuat dan Memproses Form

Bab ini menunjukkan cara mensimulasikan form kontak sederhana dan menguraikan (parse) pesan yang dikirim pengguna ke dalam sebuah struct di Go.

*forms.go*

```
package main

import (
    "html/template"
    "net/http"
)

type ContactDetails struct {
    Email    string
    Subject  string
    Message  string
}

func main() {
    tpl := template.Must(template.ParseFiles("forms.html"))

    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodPost {
            tpl.Execute(w, nil)
            return
        }

        details := ContactDetails{
            Email:    r.FormValue("email"),
            Subject: r.FormValue("subject"),
            Message: r.FormValue("message"),
        }

        // lakukan sesuatu dengan data details di sini
        _ = details

        tpl.Execute(w, struct{ Success bool }{true})
    })

    http.ListenAndServe(":8080", nil)
}
```

Jika permintaan yang masuk bukan POST, kita cukup menampilkan form kosong. Namun jika metodenya POST, kita mengambil nilai-nilai dari form menggunakan `r.FormValue`, menyimpannya ke dalam struct `ContactDetails`, lalu menampilkan halaman ucapan terima kasih.

*forms.html*

```
<!-- forms.html -->
{{if .Success}}
```

```
<h1>Thanks for your message!</h1>
{{else}}
<h1>Contact</h1>
<form method="POST">
  <label>Email:</label><br />
  <input type="text" name="email"><br />
  <label>Subject:</label><br />
  <input type="text" name="subject"><br />
  <label>Message:</label><br />
  <textarea name="message"></textarea><br />
  <input type="submit">
</form>
{{end}}
```

Jalankan programnya dengan:

```
$ go run forms.go
```

**Catatan:** Template `forms.html` memakai satu variabel `Success` untuk menentukan apakah akan menampilkan form isian atau pesan terima kasih. Pola ini — satu template dengan dua tampilan berbeda tergantung kondisi — sangat umum dipakai dalam aplikasi web sederhana.



## Bab 9. Middleware Dasar

Bab ini menunjukkan cara membuat middleware logging sederhana di Go. Sebuah middleware pada dasarnya hanyalah fungsi yang menerima `http.HandlerFunc` sebagai salah satu parameternya, membungkusnya, lalu mengembalikan `http.HandlerFunc` baru untuk dipanggil oleh server.

*basic-middleware.go*

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func logging(f http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        log.Println(r.URL.Path)
        f(w, r)
    }
}

func foo(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "foo")
}

func bar(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "bar")
}

func main() {
    http.HandleFunc("/foo", logging(foo))
    http.HandleFunc("/bar", logging(bar))

    http.ListenAndServe(":8080", nil)
}
```

Fungsi `logging` menerima sebuah handler `f`, dan mengembalikan handler baru yang lebih dulu mencatat path dari permintaan yang masuk, sebelum akhirnya memanggil handler asli `f`. Dengan cara ini, kita bisa menambahkan perilaku logging ke handler mana pun tanpa mengubah isi handler tersebut.

Jika dijalankan dan diakses beberapa kali, Anda akan melihat catatan log seperti berikut:

```
$ go run basic-middleware.go
2017/02/10 23:59:34 /foo
2017/02/10 23:59:35 /bar
2017/02/10 23:59:36 /foo?bar

$ curl -s http://localhost:8080/foo
```

```
$ curl -s http://localhost:8080/bar  
$ curl -s http://localhost:8080/foo?bar
```

## Bab 10. Middleware Tingkat Lanjut

Bab ini menunjukkan cara membuat versi middleware yang lebih canggih di Go. Middleware itu sendiri pada dasarnya hanyalah fungsi yang menerima sebuah `http.HandlerFunc` sebagai salah satu parameternya, membungkusnya, lalu mengembalikan `http.HandlerFunc` baru yang akan dipanggil oleh server.

Di sini kita mendefinisikan sebuah tipe baru bernama `Middleware`, yang nantinya memudahkan kita untuk menggabungkan (chaining) beberapa middleware sekaligus. Ide ini terinspirasi dari sebuah pembahasan tentang membangun API oleh Mat Ryer.

Potongan kode berikut menjelaskan secara rinci bagaimana sebuah middleware baru dibuat. Pada contoh lengkap di bawah, kita menyederhanakan versi ini dengan mengurangi sebagian kode boilerplate.

```
func createNewMiddleware() Middleware {  
  
    // Membuat Middleware baru  
    middleware := func(next http.HandlerFunc) http.HandlerFunc {  
  
        // Mendefinisikan http.HandlerFunc yang nantinya dipanggil oleh server  
        handler := func(w http.ResponseWriter, r *http.Request) {  
  
            // ... lakukan hal-hal middleware di sini  
  
            // Memanggil middleware/handler berikutnya dalam rantai  
            next(w, r)  
        }  
  
        // Mengembalikan handler yang baru dibuat  
        return handler  
    }  
  
    // Mengembalikan middleware yang baru dibuat  
    return middleware  
}
```

Berikut adalah contoh lengkapnya:

*advanced-middleware.go*

```
package main  
  
import (  
    "fmt"  
    "log"  
    "net/http"  
    "time"  
)
```

```

type Middleware func(http.HandlerFunc) http.HandlerFunc

// Logging mencatat setiap permintaan beserta path dan waktu prosesnya
func Logging() Middleware {

    // Membuat Middleware baru
    return func(f http.HandlerFunc) http.HandlerFunc {

        // Mendefinisikan http.HandlerFunc
        return func(w http.ResponseWriter, r *http.Request) {

            // Melakukan hal-hal middleware
            start := time.Now()
            defer func() { log.Println(r.URL.Path, time.Since(start)) }()

            // Memanggil middleware/handler berikutnya dalam rantai
            f(w, r)
        }
    }
}

// Method memastikan url hanya bisa diminta dengan metode tertentu, jika tidak akan
mengembalikan 400 Bad Request
func Method(m string) Middleware {

    // Membuat Middleware baru
    return func(f http.HandlerFunc) http.HandlerFunc {

        // Mendefinisikan http.HandlerFunc
        return func(w http.ResponseWriter, r *http.Request) {

            // Melakukan hal-hal middleware
            if r.Method != m {
                http.Error(w, http.StatusText(http.StatusBadRequest), http.StatusBadRequest)
                return
            }

            // Memanggil middleware/handler berikutnya dalam rantai
            f(w, r)
        }
    }
}

// Chain menerapkan beberapa middleware ke sebuah http.HandlerFunc
func Chain(f http.HandlerFunc, middlewares ...Middleware) http.HandlerFunc {
    for _, m := range middlewares {
        f = m(f)
    }
    return f
}

func Hello(w http.ResponseWriter, r *http.Request) {

```

```
    fmt.Fprintln(w, "hello world")
}

func main() {
    http.HandleFunc("/", Chain(Hello, Method("GET"), Logging()))
    http.ListenAndServe(":8080", nil)
}
```

Perhatikan bagaimana Chain menggabungkan handler Hello dengan middleware Method("GET") dan Logging(), sehingga setiap permintaan yang masuk akan divalidasi metodenya terlebih dahulu, lalu dicatat waktu prosesnya, sebelum akhirnya diteruskan ke handler Hello.

```
$ go run advanced-middleware.go
2017/02/11 00:34:53 / 0s

$ curl -s http://localhost:8080/
hello world

$ curl -s -XPOST http://localhost:8080/
Bad Request
```

## Bab 11. Session dan Cookie

Bab ini menunjukkan cara menyimpan data dalam session cookie menggunakan paket populer `gorilla/sessions` di Go.

Cookie adalah potongan data kecil yang disimpan di browser pengguna dan dikirimkan ke server kita pada setiap permintaan. Di dalamnya, kita bisa menyimpan misalnya status login seorang pengguna, dan mengetahui siapa dirinya (dalam sistem kita).

Pada contoh ini, kita hanya akan mengizinkan pengguna yang sudah terautentikasi untuk melihat pesan rahasia pada halaman `/secret`. Untuk mendapatkan akses, pengguna harus terlebih dahulu mengunjungi `/login` untuk mendapatkan session cookie yang valid, yang menandakan ia sudah login. Selain itu, ia juga bisa mengunjungi `/logout` untuk mencabut aksesnya ke pesan rahasia tersebut.

*sessions.go*

```
package main

import (
    "fmt"
    "net/http"

    "github.com/gorilla/sessions"
)

var (
    // key harus berukuran 16, 24, atau 32 byte (AES-128, AES-192, atau AES-256)
    key = []byte("super-secret-key")
    store = sessions.NewCookieStore(key)
)

func secret(w http.ResponseWriter, r *http.Request) {
    session, _ := store.Get(r, "cookie-name")

    // Periksa apakah pengguna sudah terautentikasi
    if auth, ok := session.Values["authenticated"].(bool); !ok || !auth {
        http.Error(w, "Forbidden", http.StatusForbidden)
        return
    }

    // Menampilkan pesan rahasia
    fmt.Fprintln(w, "The cake is a lie!")
}

func login(w http.ResponseWriter, r *http.Request) {
    session, _ := store.Get(r, "cookie-name")

    // Proses autentikasi dilakukan di sini
    // ...
}
```

```

    // Menandai pengguna sebagai sudah terautentikasi
    session.Values["authenticated"] = true
    session.Save(r, w)
}

func logout(w http.ResponseWriter, r *http.Request) {
    session, _ := store.Get(r, "cookie-name")

    // Mencabut status autentikasi pengguna
    session.Values["authenticated"] = false
    session.Save(r, w)
}

func main() {
    http.HandleFunc("/secret", secret)
    http.HandleFunc("/login", login)
    http.HandleFunc("/logout", logout)

    http.ListenAndServe(":8080", nil)
}

```

Jika Anda mencoba mengakses /secret sebelum login, Anda akan mendapatkan pesan Forbidden. Setelah mengunjungi /login, server akan mengirimkan sebuah session cookie yang bisa dipakai untuk mengakses /secret dengan sukses:

```

$ go run sessions.go

$ curl -s http://localhost:8080/secret
Forbidden

$ curl -s -I http://localhost:8080/login
Set-Cookie: cookie-name=MTQ4NzE5Mz...

$ curl -s --cookie "cookie-name=MTQ4NzE5Mz..." http://localhost:8080/secret
The cake is a lie!

```

**Catatan:** Kunci (key) yang dipakai untuk mengenkripsi cookie harus dijaga kerahasiaannya dan sebaiknya tidak ditulis langsung di dalam kode seperti pada contoh ini. Pada aplikasi nyata, simpan kunci semacam ini di environment variable atau berkas konfigurasi yang aman.

## Bab 12. Bekerja dengan JSON

Bab ini menunjukkan cara meng-encode dan men-decode data JSON menggunakan paket `encoding/json`.

*json.go*

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

type User struct {
    Firstname string `json:"firstname"`
    Lastname  string  `json:"lastname"`
    Age       int    `json:"age"`
}

func main() {
    http.HandleFunc("/decode", func(w http.ResponseWriter, r *http.Request) {
        var user User
        json.NewDecoder(r.Body).Decode(&user)

        fmt.Fprintf(w, "%s %s is %d years old!", user.Firstname, user.Lastname, user.Age)
    })

    http.HandleFunc("/encode", func(w http.ResponseWriter, r *http.Request) {
        peter := User{
            Firstname: "John",
            Lastname:  "Doe",
            Age:      25,
        }

        json.NewEncoder(w).Encode(peter)
    })

    http.ListenAndServe(":8080", nil)
}
```

Pada endpoint `/decode`, kita membaca body permintaan yang berisi JSON dan menguraikannya (decode) ke dalam struct `User`. Pada endpoint `/encode`, kita melakukan sebaliknya: mengambil sebuah struct `User` dan mengubahnya (encode) menjadi JSON untuk dikirim sebagai respons.

```
$ go run json.go

$ curl -s -XPOST -d '{"firstname":"Elon","lastname":"Musk","age":48}'
http://localhost:8080/decode
Elon Musk is 48 years old!
```



```
$ curl -s http://localhost:8080/encode  
{ "firstname": "John", "lastname": "Doe", "age": 25 }
```

**Catatan:** Tag seperti ``json:"firstname"`` di belakang setiap field struct menentukan nama key yang dipakai saat data diubah menjadi JSON atau sebaliknya. Tanpa tag ini, Go akan memakai nama field aslinya (misalnya `Firstname`, bukan `firstname`).

## Bab 13. WebSocket

Bab ini menunjukkan cara bekerja dengan WebSocket di Go. Kita akan membangun sebuah server sederhana yang mengembalikan (echo) kembali setiap pesan yang kita kirimkan kepadanya.

Untuk ini, kita perlu memasang pustaka populer gorilla/websocket dengan perintah go get berikut:

```
go get github.com/gorilla/websocket
```

Setelah terpasang, setiap aplikasi yang kita tulis nantinya bisa memanfaatkan pustaka ini.

*websockets.go*

```
package main

import (
    "fmt"
    "net/http"

    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    ReadBufferSize: 1024,
    WriteBufferSize: 1024,
}

func main() {
    http.HandleFunc("/echo", func(w http.ResponseWriter, r *http.Request) {
        conn, _ := upgrader.Upgrade(w, r, nil) // error diabaikan demi kesederhanaan

        for {
            // Membaca pesan dari browser
            msgType, msg, err := conn.ReadMessage()
            if err != nil {
                return
            }

            // Menampilkan pesan ke konsol
            fmt.Printf("%s sent: %s\n", conn.RemoteAddr(), string(msg))

            // Mengirim kembali pesan ke browser
            if err = conn.WriteMessage(msgType, msg); err != nil {
                return
            }
        }
    })

    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        http.ServeFile(w, r, "websockets.html")
    })
}
```

```
    http.ListenAndServe(":8080", nil)
}
```

Handler /echo pertama-tama meng-upgrade koneksi HTTP biasa menjadi koneksi WebSocket menggunakan `upgrader.Upgrader`. Setelah itu, ia masuk ke sebuah perulangan tak terbatas yang terus membaca pesan dari klien dan langsung mengirimkannya kembali (echo).

*websockets.html*

```
<!-- websockets.html -->
<input id="input" type="text" />
<button onclick="send()">Send</button>
<pre id="output"></pre>
<script>
    var input = document.getElementById("input");
    var output = document.getElementById("output");
    var socket = new WebSocket("ws://localhost:8080/echo");

    socket.onopen = function () {
        output.innerHTML += "Status: Connected\n";
    };

    socket.onmessage = function (e) {
        output.innerHTML += "Server: " + e.data + "\n";
    };

    function send() {
        socket.send(input.value);
        input.value = "";
    }
</script>
```

Setelah dijalankan dan Anda mengirim pesan lewat halaman web, Anda akan melihat log di terminal seperti berikut:

```
$ go run websockets.go
[127.0.0.1]:53403 sent: Hello Go Web Examples, you're doing great!
```

## Bab 14. Hashing Kata Sandi dengan bcrypt

Bab ini menunjukkan cara melakukan hashing kata sandi menggunakan bcrypt. Menyimpan kata sandi pengguna dalam bentuk polos (plain text) sangat berbahaya — jika basis data Anda bocor, seluruh kata sandi pengguna akan langsung terekspos. Karena itu, kata sandi harus selalu di-hash sebelum disimpan.

Untuk ini, kita perlu memasang pustaka bcrypt bawaan Go dengan perintah go get berikut:

```
go get golang.org/x/crypto/bcrypt
```

Setelah terpasang, setiap aplikasi yang kita tulis nantinya bisa memanfaatkan pustaka ini.

*passwords.go*

```
package main

import (
    "fmt"

    "golang.org/x/crypto/bcrypt"
)

func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), 14)
    return string(bytes), err
}

func CheckPasswordHash(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}

func main() {
    password := "secret"
    hash, _ := HashPassword(password) // error diabaikan demi kesederhanaan

    fmt.Println("Password:", password)
    fmt.Println("Hash:    ", hash)

    match := CheckPasswordHash(password, hash)
    fmt.Println("Match:   ", match)
}
```

Fungsi HashPassword mengubah kata sandi asli menjadi hash yang aman untuk disimpan di basis data. Fungsi CheckPasswordHash dipakai saat pengguna login, untuk memeriksa apakah kata sandi yang dimasukkan cocok dengan hash yang tersimpan — tanpa pernah perlu menyimpan kata sandi asli di mana pun.

```
$ go run passwords.go
```

```
Password: secret  
Hash:      $2a$14$ajq8Q7fbtFRQvXpdCq7Jcuy.Rx1h/L4J600tx.gyNLbAYctGMJ9tK  
Match:     true
```

**Catatan:** Angka 14 pada `bcrypt.GenerateFromPassword` adalah 'cost factor' yang menentukan seberapa berat proses hashing dilakukan. Semakin tinggi angkanya, semakin aman namun semakin lambat prosesnya. Nilai 10-14 umumnya sudah cukup baik untuk kebanyakan aplikasi.

## Penutup

Selamat! Anda telah menyelesaikan seluruh materi dalam buku ini, mulai dari membuat server web paling sederhana hingga topik-topik penting seperti routing, basis data, template, middleware, session, JSON, WebSocket, dan keamanan kata sandi.

Setiap contoh kode dalam buku ini sengaja dibuat ringkas agar mudah dipahami, namun tetap mencerminkan praktik yang umum dipakai dalam aplikasi Go di dunia nyata. Langkah selanjutnya yang disarankan adalah mencoba menggabungkan beberapa topik ini menjadi satu aplikasi web yang utuh — misalnya aplikasi catatan sederhana yang memiliki form, menyimpan data ke MySQL, menggunakan session untuk login, dan menyajikan halaman lewat template HTML.

Semoga buku ini membantu Anda memulai perjalanan sebagai pengembang web menggunakan bahasa Go. Selamat mencoba dan selamat berkarya!